

Databases 2 - exam - January 24, 2024 - Dur. 2h

S. Comai, P. Fraternali, D. Martinenghi

Students with 30% reduction can omit:

TS (Mono and Multi) in Ex. A and Questions 2 and 3 in Ex. B.

A. Concurrency control (12 points, up to 2 points per class)

Consider the schedule $S = w_3(x) r_2(y) r_1(x) w_3(z) w_2(x) r_3(y) w_1(x) w_1(z)$. Classify S with respect to VSR (in case, listing the view equivalent schedules), CSR, 2PL, Strict 2PL, TS Mono, and TS Multi (with the conventions adopted for TS Multi under Snapshot Isolation, used for the exercises). Justify your answers exhaustively but concisely (determine membership using class inclusion wherever possible).

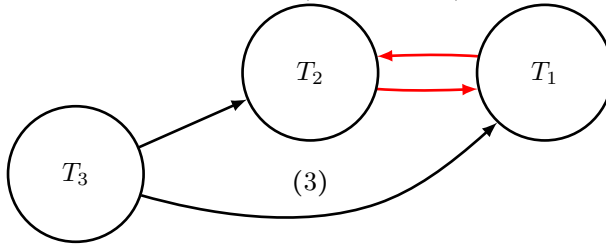
Solution.

Overview

S is VSR. S is not CSR, and thus also not 2PL, not Strict 2PL, and not TS Mono for the class inclusion relationships. S is not TS Multi (under Snapshot Isolation).

Detailed analysis for the CSR class

The resource graph is cyclic (cycles in red).



Detailed analysis for the VSR class

Projection on resources:

x	w_3	r_1	w_2	w_1	
y		r_2		r_3	
z			w_3		w_1

Set of reads-from: $\{w_3(x) \rightarrow r_1(x)\}$

Set of final writes: $\{w_1(x), w_1(z)\}$

Set of blind writes: $\{w_2(x), w_3(z)\}$

The schedule is not CSR but the blind write $w_2(x)$ can be moved before $w_3(x)$, thereby remaining blind and preserving the same reads-from and final writes.

View-equivalent to serial schedule: $r_2(y) w_2(x) w_3(x) w_3(z) r_3(y) r_1(x) w_1(x) w_1(z)$.

Detailed analysis for the TS Multi class

Operation	RTM(x)	WTM(x)	RTM(y)	WTM(y)	RTM(z)	WTM(z)	Killed transactions
$w_3(x)$	0 (0)	0, 3	0 (0)	0	0 (0)	0	
$r_2(y)$			2 (0)				
$r_1(x)$	1 (0)						
$w_3(z)$						0, 3	
$w_2(x)$							2 as $2 < \text{last WTM}(x) = 3$
$r_3(y)$			3 (0)				
$w_1(x)$							1 as $1 < \text{last WTM}(x) = 3$
$w_1(z)$							1 (already killed)

Check if 30% reduction: ☐

Last name: _____

First name: _____

Pers. code: _____

Signature: _____

B. Ranking (10 points)

Consider the following three ranked lists of travel destinations (each entry is in the form *destination: score*)

Landscape	Cultural experience	Availability of activities
<i>A</i> : 0.9	<i>B</i> : 0.9	<i>F</i> : 1.0
<i>B</i> : 0.8	<i>D</i> : 0.8	<i>D</i> : 0.9
<i>C</i> : 0.8	<i>F</i> : 0.8	<i>G</i> : 0.9
<i>D</i> : 0.7	<i>A</i> : 0.6	<i>A</i> : 0.8
<i>E</i> : 0.6	<i>C</i> : 0.6	<i>B</i> : 0.7
<i>F</i> : 0.6	<i>G</i> : 0.5	<i>E</i> : 0.6
<i>G</i> : 0.5	<i>E</i> : 0.5	<i>C</i> : 0.5

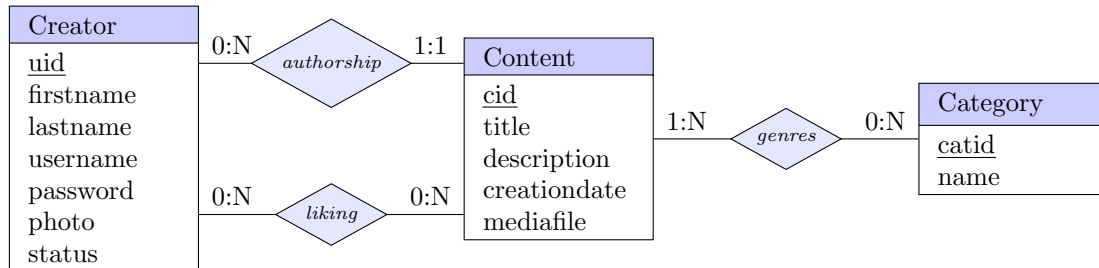
1. Determine the top-2 travel destinations using Fagin's Algorithm (FA) (4 points). The user is interested in exploring two scoring functions: one based on the maximum of the three scores, and the other based on the minimum of the three scores. Quantify the number of sorted accesses (s.a.) and random accesses (r.a.) required to obtain the requested results. In both cases, the top destinations are the ones with the highest scoring function values.
2. Same as the previous point, but with the Threshold Algorithm (TA) (4 points).
3. TA is instance optimal. Among the algorithms discussed in the lectures, are there any algorithms that incur lower costs than TA in terms of s.a. + r.a., specifically for this particular instance? (2 points)

Solution.

See separate pdf file.

C. JPA (10 points)

An application manages multimedia user-generated content. Content items have a title, a description, a media file, and a creation date and are associated with one or more categories. Creators may “like” content by other creators. The application lets creators log in and access the list of their created contents, ordered by descending creation date. They can select a category and view the contents associated with it ordered by descending creation date. For each content, they can see the creator and the list of creators who liked it, in descending alphabetical order of the surname. Creators are in the order of millions, contents in the order of tens of millions, and categories in the order of tens. Contents are uniformly distributed over the creators who produce them and over the categories.



1. Design the annotations for the relationships in the conceptual model. Specify the owner side of each relationship, define the appropriate 'mapped-by' attribute, and detail the fetch policy and cascading policies. Consider and justify the choices made in the design to support the required access patterns of the web application. (5 points)
2. Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above-mentioned access paths of the application and data cardinalities. (5 points)

Solution.

See separate pdf file.

Consider the following three ranked lists of travel destinations (each entry is in the form *destination_id: score*)

1) Determine the top-2 travel destinations using Fagin’s Algorithm (FA) (4 points) and the Threshold Algorithm (TA) (4 points). The user is interested in exploring two scoring functions: one based on the maximum of the three scores, and the other based on the minimum of the three scores. Quantify the number of sorted accesses (s.a.) and random accesses (r.a.) required to obtain the requested results.
In both cases, the top destinations are the ones with the highest scoring function values.

2) TA is instance optimal. Among the algorithms discussed in the lectures, are there any algorithms that incur lower costs than TA in terms of s.a. + r.a., specifically for this particular instance? (2 points)

Landscape	Cultural experiences	Availability of activities
A: 0.9	B: 0.9	F: 1.0
B: 0.8	D: 0.8	D: 0.9
C: 0.8	F: 0.8	G: 0.9
D: 0.7	A: 0.6	A: 0.8
E: 0.6	C: 0.6	B: 0.7
F: 0.6	G: 0.5	E: 0.6
G: 0.5	E: 0.5	C: 0.5

Fagin Algorithm

12 s.a.
+ 6 r.a.

Landscape	Cultural experiences	Availability of activities
A: 0.9	B: 0.9	F: 1.0
B: 0.8	D: 0.8	D: 0.9
C: 0.8	F: 0.8	G: 0.9
D: 0.7	A: 0.6	A: 0.8
E: 0.6	C: 0.6	B: 0.7
F: 0.6	G: 0.5	E: 0.6
G: 0.5	E: 0.5	C: 0.5

	Max	Min
A	0.9	0.6
B	0.9	0.7
C	0.8	0.5
D	0.9	0.7
F	1.0	0.6
G	0.9	0.5

Top 2 - max	Score
F	1.0
any two among A, B, D, G, chosen non-deterministically	0.9

Top 2 - min	Score
B	0.7
D	0.7

12 s.a. + 6 r.a. to get both results

Threshold Algorithm

Step 1:
3 s.a.
+ 6 r.a.

Landscape	Cultural experiences	Availability of activities
A: 0.9	B: 0.9	F: 1.0
B: 0.8	D: 0.8	D: 0.9
C: 0.8	F: 0.8	G: 0.9
D: 0.7	A: 0.6	A: 0.8
E: 0.6	C: 0.6	B: 0.7
F: 0.6	G: 0.5	E: 0.6
G: 0.5	E: 0.5	C: 0.5

Top 2 max	Score
F	1.0
A or B chosen non- determini- stically	0.9

Threshold max
value: $T = 1.0$
point: $\tau = (0.9, 0.9, 1.0)$

Top 2 min	Score
B	0.7
A or F chosen non- determini- stically	0.6

Threshold min
value: $T = 0.9$
point: $\tau = (0.9, 0.9, 1.0)$

Threshold Algorithm

Step 2:
3 s.a.
+ 1 r.a.

Landscape	Cultural experiences	Availability of activities
A: 0.9	B: 0.9	F: 1.0
B: 0.8	D: 0.8	D: 0.9
C: 0.8	F: 0.8	G: 0.9
D: 0.7	A: 0.6	A: 0.8
E: 0.6	C: 0.6	B: 0.7
F: 0.6	G: 0.5	E: 0.6
G: 0.5	E: 0.5	C: 0.5

Top 2 max	Score
F	1.0
A or B or D chosen non-deterministically	0.9

Top 2 min	Score
B	0.7
D	0.7

Threshold max
value: $T = 0.9$
point: $\tau = (0.8, 0.8, 0.9)$

Threshold min
value: $T = 0.8$
point: $\tau = (0.8, 0.8, 0.9)$

For max we can stop
after 6 s.a. and 7.r.a.

Threshold Algorithm

Step 3:
3 s.a.
+ 4 r.a.

Landscape	Cultural experiences	Availability of activities
A: 0.9	B: 0.9	F: 1.0
B: 0.8	D: 0.8	D: 0.9
C: 0.8	F: 0.8	G: 0.9
D: 0.7	A: 0.6	A: 0.8
E: 0.6	C: 0.6	B: 0.7
F: 0.6	G: 0.5	E: 0.6
G: 0.5	E: 0.5	C: 0.5

Top 2 min	Score
B	0.7
D	0.7

Threshold min
value: $T = 0.8$
point: $\tau = (0.8, 0.8, 0.9)$

Threshold Algorithm

Step 4:
3 s.a.

Landscape	Cultural experiences	Availability of activities
A: 0.9	B: 0.9	F: 1.0
B: 0.8	D: 0.8	D: 0.9
C: 0.8	F: 0.8	G: 0.9
D: 0.7	A: 0.6	A: 0.8
E: 0.6	C: 0.6	B: 0.7
F: 0.6	G: 0.5	E: 0.6
G: 0.5	E: 0.5	C: 0.5

Top 2 min	Score
B	0.7
D	0.7

Threshold min
value: $T = 0.6$
point: $\tau = (0.7, 0.6, 0.8)$

For min we can stop
after 12 s.a. and 11.r.a.

- For max, B0 performs better: to get the top-2 travel destinations, it needs only $3*2=6$ s.a.
- For min, FA required less s.a.+r.a than TA

Data bases 2

Written exam 2024: JPA exercise

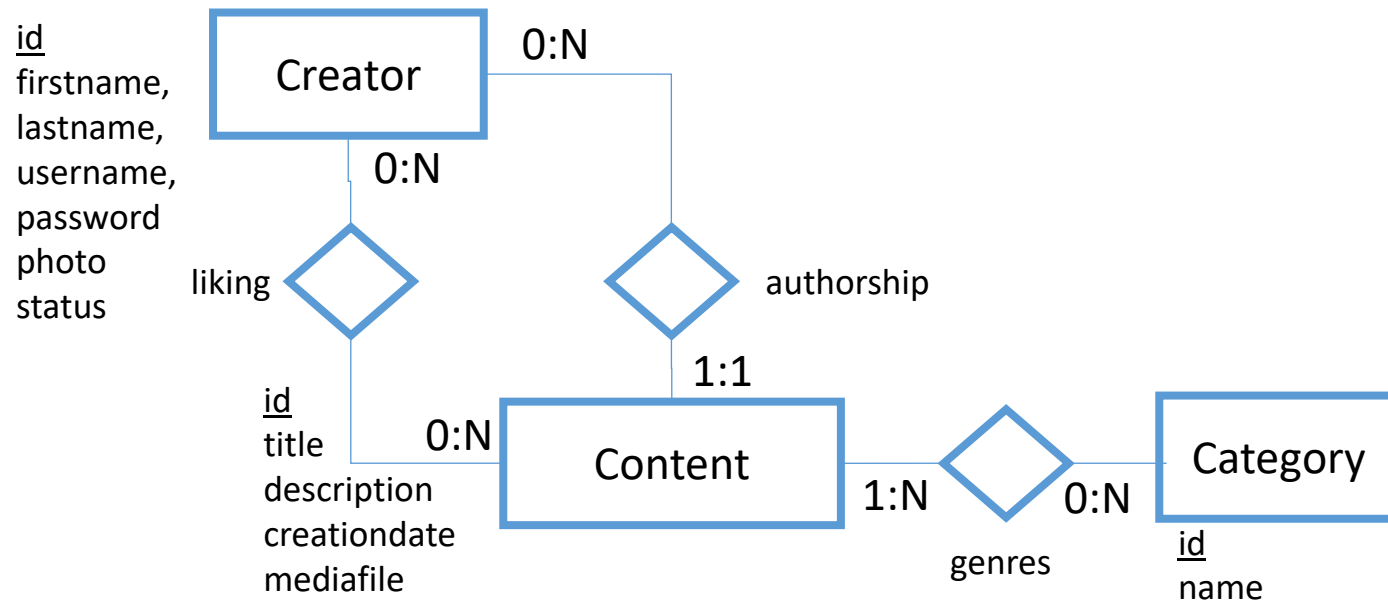
Specifications

- An application manages multimedia user generated content. Content items have a title, a description, a media file, and a creation date and are associated with one or more categories. Creators may “like” contents by other creators. The application lets creators log in and access the list of their created contents, ordered by descending creation date. They can select a category and view the contents associated to it ordered by descending creation date. For each content, they can see the creator and the list of creators who liked it, in descending alphabetical order of the surname. Creators are in the order of millions, contents in the order of tens of millions, and categories in the orders of tens. Contents are uniformly distributed over the creators who produce them and over the categories.

Questions

- Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application.
- Comment the design choices you have made.

Entity Relationship



Relational model

Content(CID, title, description, creationdate, mediafile, author)

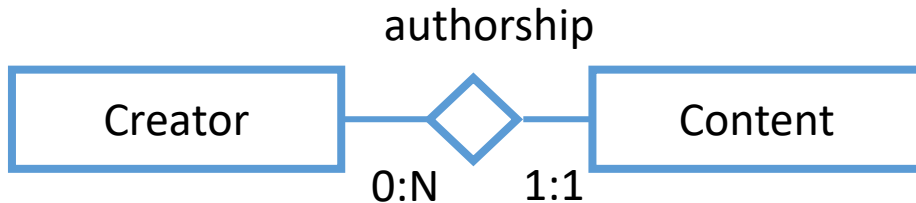
Liking(CID, UID)

Creator(UID, firstname, lastname, photo, status, username, password)

Genres(CID, CATID)

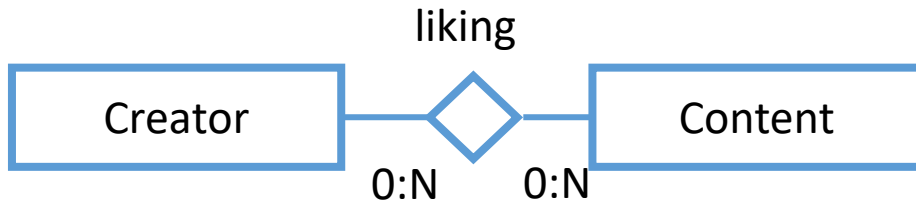
Category(CATID, name)

Relationship “authorship”



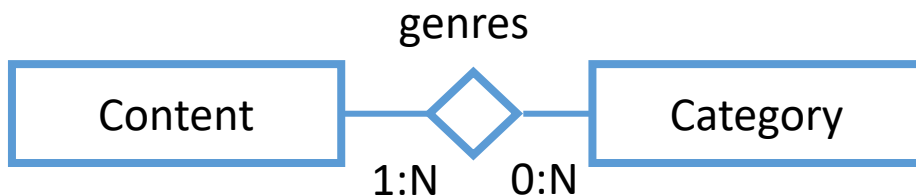
- Creator → Content
@OneToMany is necessary to get the contents created by the Creator
 - @OrderBy(“creationdate DESC”)
 - Owner = Content
 - FetchType.EAGER because the contents per creator are in the order of tens on average and the media files are not loaded eagerly
- Content → Creator
@ManyToOne is requested to show the creator

Relationship “liking”



- Content → Creator
@ManyToOne is necessary to show the creators that liked a content
 - @OrderBy(“lastname DESC”)
 - Owner = either Creator or Content
 - FetchType.LAZY because the creators who liked the item may be many (no statistics are given)
- Creator → Content
@ManyToOne is not requested, but shown for simplicity
 - FetchType.LAZY because the items liked by a creator may be many (no statistics are given)

Relationship “genres”



- Category → Content
@ManyToMany is necessary to show the contents of a Category
 - Owner = either Content or Category
 - FetchType.LAZY because the number of contents per category may be many
 - @OrderBy("creationdate DESC")
- Content → Category
@ManyToMany not requested, but mapped for simplicity
 - FetchType.EAGER because a content may be associated to very few categories

Entity Creator

```
@Entity
public class Creator implements Serializable {
    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int uid;

    private String firstname;
    private String lastname;
    private String username;
    private String password;

    @Basic(fetch = FetchType.LAZY)
    @Lob
    private byte[] photo;
    private String status;

    @OneToMany(mappedBy = "author", fetch = FetchType.EAGER)
    @OrderBy("creationdate DESC")
    private List<Content> authoredContents;

    @ManyToMany(mappedBy = "likers", fetch = FetchType.LAZY)
    private List<Content> likedContents;
```

Entity Content

```
@Entity
public class Content implements Serializable {
    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int cid;
    private String title;
    private String description;
    @Temporal(TemporalType.DATE)
    private Date creationdate;
    @Basic(fetch = FetchType.LAZY)
    @Lob private byte[] mediafile;

    @ManyToOne
    @JoinColumn(name="author")
    private Creator author; //owner

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(name="liking",
        joinColumns={@JoinColumn(name="cid")},
        inverseJoinColumns={@JoinColumn(name="uid")}
    )
    @OrderBy("lastname DESC")
    private List<Creator> likers; //owner

    @ManyToMany(mappedBy = "items", fetch = FetchType.EAGER)
    private List<Category> categories;
```

Entity Category

```
@Entity
public class Category implements Serializable {
    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int catid;
    private String name;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name="genres",
        joinColumns={@JoinColumn(name="catid")},
        inverseJoinColumns={@JoinColumn(name="cid")}
    )
    @OrderBy("creationdate DESC")
    private List<Content> items; // owner
```